AI VIDEO STUDIO

Project Overview & Developer Guide

Repository	github.com/Malathiale/ai-video-studio
Project type	Image-to-video web application
Model integration	Wan 2.2 hosted inference through Hugging Face Gradio

A concise guide to the application, its structure, local setup, generation workflow, and repository contents.

1. Project at a glance

AI Video Studio is a web application intended to turn a still image and a text prompt into a short generated video. The user-facing interface collects the source image and generation instructions, while the Python backend coordinates the request to a hosted Wan 2.2 image-to-video model.

Core user workflow

- Open the studio in a web browser.
- Choose or upload a source image.
- Write a prompt describing the desired motion or scene.
- Adjust available generation options and submit the request.
- Wait for the hosted model to process the request, then preview the generated result.
- Use the gallery to review saved creations, if that feature is enabled in the running app.

Important service note

Video generation depends on the availability, usage limits, and queue capacity of the external hosted inference service. A temporary quota or GPU-capacity error can prevent a generation even when the local web application is running correctly.

2. Technology and architecture

The repository contains a Python application layer, browser-side assets, and a model-integration module. The exact responsibilities may evolve as the project is developed.

Part	Purpose
`templates/index.html`	Page structure and interface markup.
`static/style.css`	Visual styling and responsive layout.
`static/script.js`	Browser interactions and frontend behavior.
`app.py / main.py`	Python application entry points and web routes.
`wan_engine.py`	Wan 2.2 hosted model integration and generation request handling.
`requirements.txt`	Python package dependencies.
`.gitignore`	Excludes local secrets, environments, and generated/runtime files from Git.

3. Run the project locally

The commands below assume Windows PowerShell and that Python is installed.

Open the project folder

```
cd C:\Users\YOUR_USER\AI-VIDEO-STUDIO
```

Create and activate a virtual environment (if one does not already exist)

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Install dependencies

```
pip install -r requirements.txt
```

Configure environment variables

Create a local **.env** file and add the required access token(s) for the provider configured by the application. Use the variable names expected by the code. Never paste real tokens into the README, source code, screenshots, or this PDF.

Start the application

```
python app.py
or use the startup command documented in the repository if it differs.
```

Open the local address printed by the server in your browser. In the development setup used for this project, the address has been **http://127.0.0.1:8002**.

4. Repository screenshots

The screenshots folder documents the interface and its main user-facing areas. The current repository includes these image files:

- **screenshots/studio.png** — main studio interface.
- **screenshots/upload-section.png** — image upload and prompt area.
- **screenshots/generated-video.png** — generated video preview.
- **screenshots/gallery.png** — creations gallery.

5. Git and GitHub workflow

After making changes, use the following commands from the project directory:

```
git status
git add .
git commit -m "Describe your changes"
git push
```

Review git status before committing. Confirm that private configuration files, API tokens, virtual environments, personal images, and generated files are not being added unintentionally.

6. Security and responsible use

- Keep API tokens in a local .env file and ensure .env is listed in .gitignore.
- If a token is accidentally pushed to a public repository, revoke or rotate it immediately; deleting it in a later commit does not make the exposed token safe.
- Only publish images and videos that you have permission to share. Check screenshots for private photos, personal information, and secrets.
- Follow the model host's terms, rate limits, and content policies.

7. Troubleshooting notes

Generation fails but the website opens: The web interface and hosted inference service are separate. Check the backend error message, provider availability, queue, and account quota.

Dependency installation fails: Confirm the virtual environment is active, then retry *pip install -r requirements.txt*. If the dependency list has changed, update requirements.txt after testing.

Git does not show screenshots: Confirm that the files are inside the repository's screenshots folder, then check *git status*.

8. Possible future improvements

- Add clear progress and error messages during generation.
- Validate image formats and file sizes before submission.
- Improve generation history, download controls, and gallery organization.
- Add automated tests and document deployment steps.
- Provide graceful handling for provider quota and temporary service outages.

Project repository

https://github.com/Malathiale/ai-video-studio

This guide describes the current project structure and the development workflow discussed for this repository. As the code changes, keep the setup commands and feature descriptions in sync with the actual implementation.